

Технологии программирования

Бразильский Китай



Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Азиатский Китайский Китай



Азиатский Китайский Китай

Азиатский Китайский Китай

Азиатский Китайский Китай

Азиатский Китайский Китай



Бразильский Китай

Бразильский Китай

Бразильский Китай



Бразильский Китай



Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Бразильский Китай

Тамплиеры

Древо

Наследование

Возможность порождать классы из другого с сохранением методов класса-предка, добавляя при этом свои.

Иерархия классов должна отражать структуру мира как иерархическую.

```
class Mother {  
class Father {  
};  
  
class Child : public Mother, public Father {  
//Child наследует свойства обоих родителей
```

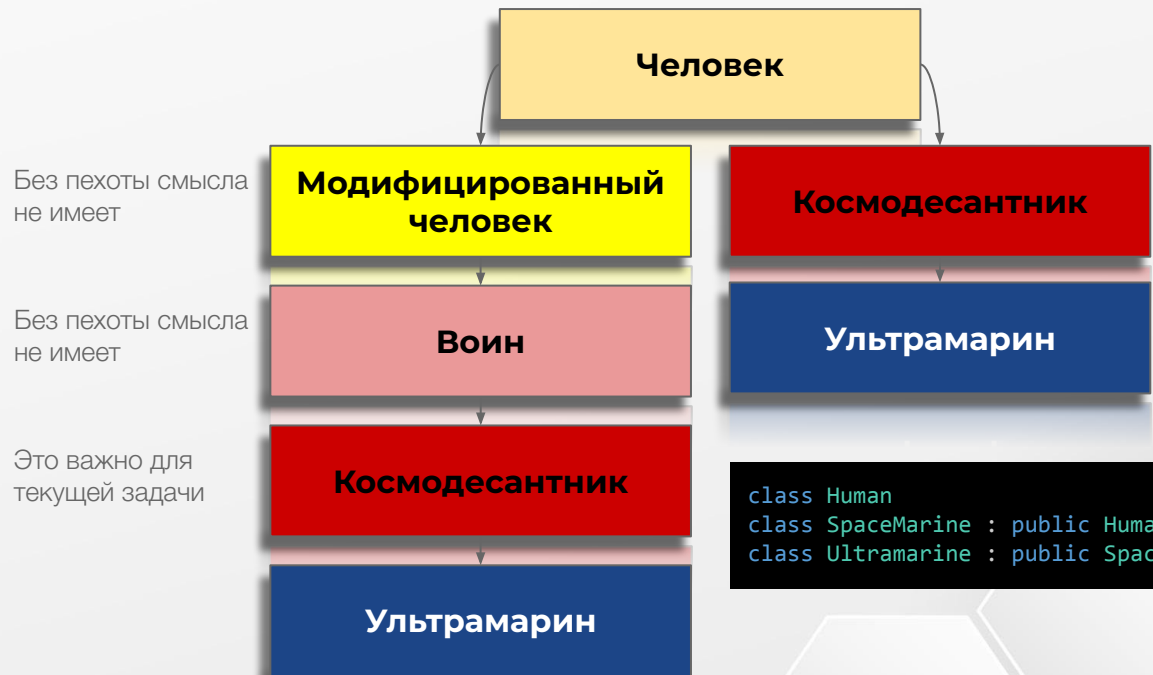




ADEPTUS ASTARTES

Ультрамарины

Иерархия наследования



```
class Human
class SpaceMarine : public Human
class Ultramarine : public SpaceMarine
```

Создавая цепь наследования учитываем абстракцию и минимизацию наследования



CELENG.DEVIANTART

Что в тебе от человека

Наследование

Наследники получают:

- public методы и поля родительских классов.
- protected методы и поля родительских классов.
- Операторы родительских классов.
- Доступ к private членам родительских классов через public/protected методы родительских классов.

Наследники не получают:

- Конструкторы - их набор уникален для каждого класса.
- Деструктор - уникален для каждого класса.
- private методы и поля.



Снаряжение

Конструкторы

- Чтобы сконструировать класс наследник необходимо вызвать конструкторы для всех классов родителей.
- В случае конструкторов по умолчанию, цепочка вызовов будет сконструирована автоматически.
- Если конструкторов по умолчанию нет, цепочка вызовов конструкторов создается вручную:

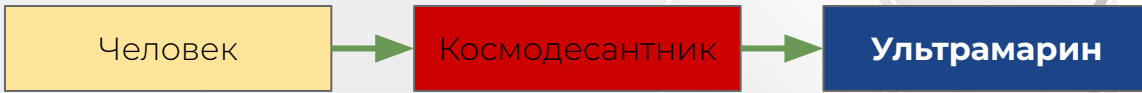
```
class Human
{
    Human(const std::string& name) : m_name(name)
    {};
```

```
class SpaceMarine : public Human
{
    SpaceMarine(const std::string& name, uint32_t rank)
        : Human(name), m_rank(rank) {};
```

- Все конструкторы в цепочке наследования делегирующие.
- Делегировать конструирование можно только своему непосредственному родителю.
- В случае наследования возможно смешивание делегирования конструктору родителя и инициализацию собственных полей.

Порядок вызова конструкторов

- Вызов конструкторов выполняется от родителей к наследникам



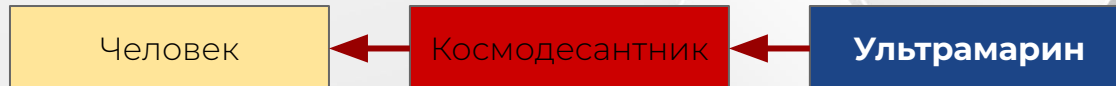
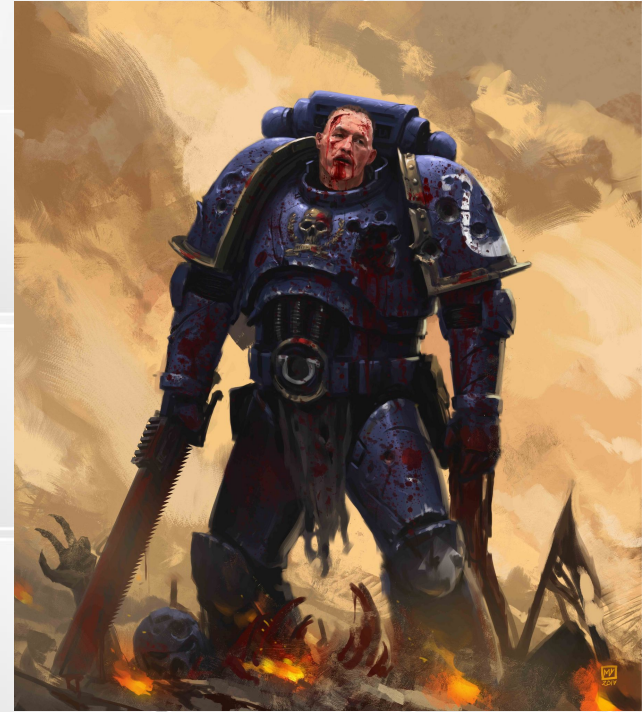
Многослойная броня

Деструктор

- Уникален для каждого класса в иерархии.
- Чтобы уничтожить все члены включая родительские, должны быть вызваны все деструкторы по цепочке.
- Управлять цепочкой вызовов нельзя.
- Деструктор родителя вызывается когда все члены класса наследника уничтожены.

Порядок вызова деструкторов

- Вызов деструкторов выполняется от классов наследников к классам родителям.



Под разными углами

Человек

Космодесантник

Ультрамарин

Upcast

- Upcast - приведение классов наследников к родительским классам.
- Upcast операторы доступны по умолчанию.
- Upcast операторы по указателю доступны по умолчанию.
- Upcast можно делать на любом уровне иерархии и к любому родительскому типу вне зависимости от глубины.

```
Ultramarine ultramarine("Marneus Calgar");  
SpaceMarine* astartes = &ultramarine; // легально  
Human* citizen      = &ultramarine;   // легально
```

Downcast

- Downcast - приведение классов родителей к типам наследников.
- По умолчанию операторы downcast отсутствуют, требуется явное приведение.

```
Ultramarine* ultramarine = static_cast<Ultramarine*>(human);
```

- Простой downcast небезопасен, существует риск приведения не к тому наследнику. Более безопасен `dynamic_cast`, в случае неверного преобразования он вернет `nullptr`.
- `dynamic_cast` требует чтобы объекты в цепочке наследования были полиморфными.

```
Ultramarine* ultramarine = dynamic_cast<Ultramarine*>(human);
```



Бессмертный Император Человечества

Запрет на наследование

Ключевое слово **final**

- Директива компилятору, о том что класс является последним в цепочке наследования.
- Финальным может быть независимый класс, который не от чего не наследуется.
- Наследоваться от final класса нельзя.
- Опционально.

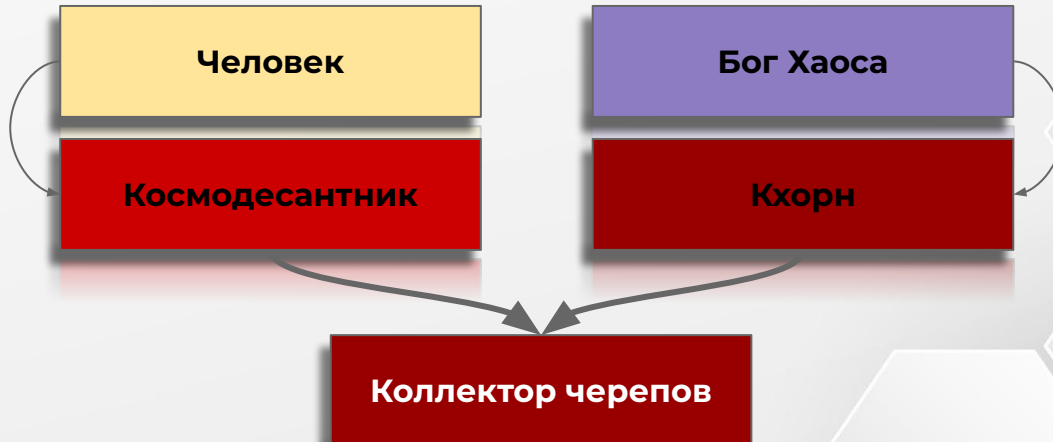
```
class Emperor final : public _GOD_
```



Космодесант Хаоса

Множественное наследование

- Класс может быть наследован от множества родителей.
- Правила конструирования и уничтожения сохраняются.
- Правила up-down cast сохраняются и работают независимо для каждой ветви наследования.



```
class SkullCollector: public SpaceMarine, public Khorn
```

Хорус (Гор) Луперкаль

Типы наследования: public, protected, private

public (по умолчанию для структур):

- Публичные унаследованные методы и поля остаются публичными.
- Доступно внешнее приведение к родительскому типу.
- Downcast извне поддерживается.

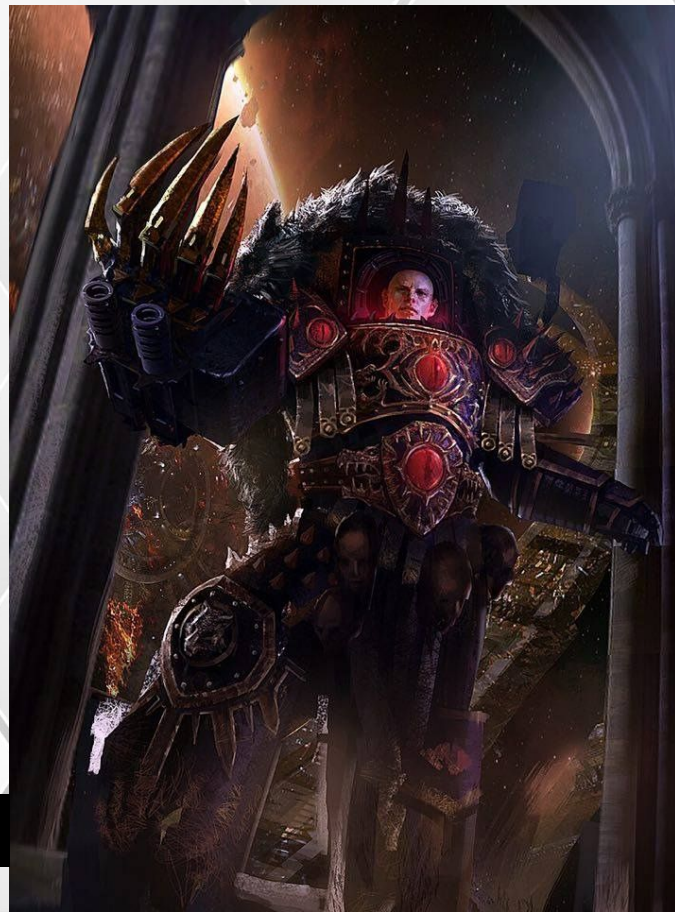
protected:

- Публичные унаследованные методы и поля становятся защищенными и доступны только классу и его наследникам.
- Приведение к родительскому типу доступно классу и его наследникам.
- Downcast доступен только классу и его наследникам.

private (по умолчанию для классов):

- Унаследованные методы и поля становятся приватными.
- Приведение к родительскому типу доступно только внутри класса.
- Downcast доступен только внутри класса.
- Снаружи класс выглядит как не наследующий.

```
class HorusLupercal : public SpaceMarine, private ChaosUndivided
```



Единой верой

Простой оверрайд

- Одинаковая сигнатура функции в классах родителях и наследниках.
- При использовании объекта наследника будет вызвана функция того типа из которого она вызвана.

```
void TellMeYourName() /*class Human*/ {  
    printf("nameless human\n");  
};
```

```
void TellMeYourName() /*class Ultramarine*/ {  
    printf("Marneus Calgar\n");  
};
```

Отличие от оверлоада

- Сигнатура оверрайдов полностью идентична.
- Оверрайд можно создать только для функций в классах связанных наследованием.

```
Human human;  
Ultramarine ultramarine("Marneus Calgar");  
  
ultramarine.TellMeYourName(); // "Marneus Calgar"  
human.TellMeYourName();      // "nameless human"  
  
Human* adept = &ultramarine; // upcast - это всё ещё Марнеус  
adept->TellMeYourName();      // "nameless human"
```



Принцип полиморфизма нарушен. В контексте родителя мы не можем работать, нам нужно знать тип объекта.

Аугментация

Полиморфизм

Возможность использовать классы потомки в контексте, который был предназначен для класса – предка.

Взаимодействие с объектами классов ведется только на уровне их интерфейсов, игнорируя внутреннее устройство. Нам достаточно интерфейса и нам всё равно что за объект за ним скрывается, важно только то, что мы можем вызвать любую функцию интерфейса и она будет выполнена в соответствии с нашими ожиданиями (теоретически...)



**Адептус Скелетор еще
вернется и всё объяснит...**